

从 C 函数到 32 位汇编：一次函数调用中的栈变化

真实姓名：姚翔

学号：20262822

实验名称：Linux 内核分析实验一

实验环境：实验楼 64 位 Linux 虚拟机中的 32 位汇编编译环境

实验日期：2026 年 9 月 24 日

原创作品转载请注明出处。

《Linux内核分析》MOOC课程 <http://mooc.study.163.com/course/USTC-1000029000>

一、实验目标

本实验使用 GCC 将 C 程序编译成 32 位 AT&T 格式汇编，观察 `main` 调用 `f`、`f` 调用 `g` 的过程，重点分析函数序言、函数尾声、参数传递、返回值以及栈指针和帧指针的变化，从而理解一条 C 语句如何落到处理器可执行的指令上。

二、源程序与编译

为避免和其他同学完全相同，我使用了如下数值：

```
int g(int x)
{
    return x + 5;
}

int f(int x)
{
    return g(x);
}

int main(void)
{
    return f(6) + 2;
}
```

编译命令为：

```
gcc -S -o main.s main.c -m32
cat main.s
```

第一次输入时把 `int` 误写成了 `nt`，编译器提示 `unknown type name`

`'nt'`；修正源程序后再次执行命令，生成汇编文件成功。截图还说明本实验采用 AT&T

语法：源操作数在前，目标操作数在后，例如 `movl 8(%ebp), %eax` 表示把栈中参数取到 `%eax`。

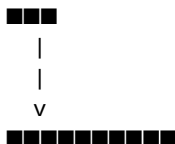
```
shiyanolou@6ab4a5d96f8c024125cd7ecc: ~
shiyanolou:~/ $ vim main.c [12:26:02]
shiyanolou:~/ $ gcc -S -o main.s main.c -m32 [12:28:08]
main.c:1:1: error: unknown type name 'nt'
nt g(int x)
^
shiyanolou:~/ $ vim main.c [12:28:28]
shiyanolou:~/ $ gcc -S -o main.s main.c -m32 [12:29:41]
shiyanolou:~/ $ [12:29:54]
```

图1：第一次编译因类型名拼写错误失败，修正为 int 后编译命令正常返回。

三、分析前需要理解的寄存器和栈

本实验重点观察 EIP、ESP、EBP 和 EAX。EIP 是指令指针，决定 CPU 下一步取哪条指令；ESP 是栈指针，指向当前栈顶；EBP 是栈帧基址寄存器，函数建立栈帧后可通过固定偏移访问参数；EAX 是通用寄存器，在本实验的调用约定中保存整数返回值。32 位 x86 栈向低地址方向增长，每次压入 4 字节数据，ESP 通常减少 4；弹出数据时 ESP 向高地址恢复。

可以把栈方向抽象为：



因此，pushl 会让 ESP 向低地址移动并写入数据，popl 会读取栈顶数据并让 ESP 向高地址移动。call 还会把返回地址压栈，ret 则取出该地址恢复控制流。

四、main 函数：建立栈帧并调用 f

main 的核心代码为：

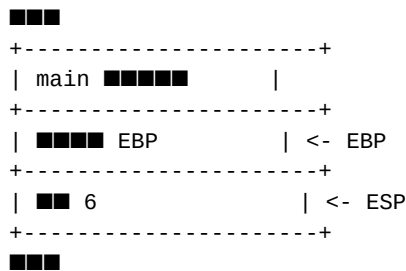
```
pushl %ebp
movl %esp, %ebp
```

```

subl $4, %esp
movl $6, (%esp)
call f
addl $2, %eax
leave
ret

```

首先, `pushl %ebp` 把进入 `main` 前的 `EBP` 保存到栈中, `ESP` 减少 4; `movl %esp,%ebp` 让 `EBP` 指向保存旧 `EBP` 的位置, 从而建立 `main` 的栈帧。随后 `subl $4,%esp` 预留一个 4 字节参数槽, `movl $6, (%esp)` 把常量 6 写入该位置。



执行 `call f` 时, CPU 把 `call` 后下一条指令的地址压入栈顶, 再跳到 `f`。这样 `f` 执行结束后, `ret` 才能准确回到 `addl $2,%eax`。

五、进入 `f`: 参数为什么位于 `8(%ebp)`

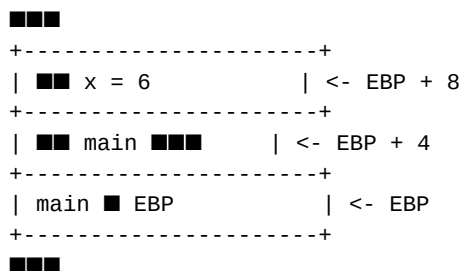
`f` 的代码为:

```

pushl %ebp
movl %esp, %ebp
subl $4, %esp
movl 8(%ebp), %eax
movl %eax, (%esp)
call g
leave
ret

```

刚进入 `f` 时, 栈中已有 `main` 准备的参数 6 和 `call f` 保存的返回地址。`f` 的序言保存 `main` 的 `EBP`, 再令 `EBP` 指向自己的栈帧:



所以 `EBP+0` 是保存的旧 `EBP`, `EBP+4` 是返回地址, `EBP+8` 是第一个参数。`movl 8(%ebp),%eax` 取出参数后得到 `EAX=6`。这正是栈帧让参数访问保持稳定的原因, 即使 `ESP` 后面因为局部空间变化, 参数仍然可以通过 `EBP` 的固定偏移找到。

六、f 调用 g 时的堆栈变化

f 执行 `subl $4,%esp`，在自己的栈帧下方预留 4 字节；再执行 `movl %eax, (%esp)`，把 EAX 中的 6 写入该空间，为 g 准备参数：

```

■■■■
+-----+
| f ■■■■ x = 6      | <- f ■ EBP + 8
+-----+
| ■■ main ■■■■      | <- f ■ EBP + 4
+-----+
| main ■ EBP         | <- f ■ EBP
+-----+
| g ■■■■ 6           | <- ESP
+-----+
■■■■
```

接着执行 `call g`。CPU 再压入一个返回地址，控制流转入 g。此时栈顶增加了“返回 f 的地址”，g 的序言会继续保存 f 的 EBP，因此每一层调用都有独立的返回点和栈帧。

七、g 函数：完成真正的加法运算

```

pushl %ebp
movl  %esp, %ebp
movl  8(%ebp), %eax
addl  $5, %eax
popl  %ebp
ret
```

g 保存 f 的 EBP 后，`8(%ebp)` 仍然按照“旧 EBP、返回地址、参数”的布局指向参数 6：

```

■■■■
+-----+
| f ■■■■ x = 6      |
+-----+
| ■■ main ■■■■      |
+-----+
| main ■ EBP         |
+-----+
| g ■■■■ x = 6      | <- g ■ EBP + 8
+-----+
| ■■ f ■■■■         | <- g ■ EBP + 4
+-----+
| f ■ EBP           | <- g ■ EBP
+-----+
■■■■
```

`movl 8(%ebp),%eax` 使 EAX=6，`addl $5,%eax` 使 EAX=11，这两条指令实现 C 代码 `return x + 5`。整数返回值放在 EAX，所以 g 返回时 EAX 中的 11 就是 g(6) 的结果。

八、g 返回 f，f 再返回 main

g 执行 `popl %ebp`, 恢复 f 的 EBP ; `ret` 弹出 call g 保存的返回地址, 回到 f 中 call g 的下一条指令。此时 `EAX=11`。f 没有修改 EAX, 随后执行 `leave` 和 `ret`。

`leave` 可以近似展开为：

```
movl %ebp, %esp
popl %ebp
```

它先丢弃 f 的临时参数空间, 再恢复 main 的 EBP ; `ret` 使用 call f 保存的返回地址回到 main。因此 f 返回给 main 的值仍是 11, 对应 C 代码 `return g(x)`。

九、回到 main 并得到最终结果

main 恢复执行 `addl $2,%eax`。此时 `EAX=11`, 所以 `EAX=11+2=13`, 对应 `return f(6)+2`。最后 main 执行 `leave`、`ret`, 撤销自己的栈帧并把返回值交给操作系统。

完整控制流是：

```
main --■■■■6--> f --■■■■6--> g
  ^               ^               |
  |               +---- EAX=11 <----+
  +----- EAX=11■■■■2■■■13
```

十、对 call、leave 和 ret 的进一步理解

`call` 同时完成保存返回地址和改变控制流；被调用函数通过 `pushl %ebp`、`movl %esp,%ebp` 建立稳定的栈帧；参数用 EBP 加偏移访问, 计算结果放入 EAX。`leave` 撤销当前栈帧并恢复调用者的 EBP, `ret`

使用栈中的返回地址恢复执行位置。它们共同解决了“去哪里执行”和“执行完回到哪里”两个问题。

十一、实验总结：我对“计算机是如何工作的”理解

在 C 语言层面, `return f(6)+2` 只是一条表达式；在汇编层面, 它包含参数准备、返回地址保存、栈帧建立、寄存器运算、控制流跳转和现场恢复。CPU 按 EIP 指向的位置取指令, 通过 ESP 和 EBP 管理调用栈, 通过 EAX 传递计算结果, 内存保存参数、返回地址和现场。`main -> f -> g -> f -> main` 这条链路说明, 函数调用并不是处理器天然理解的概念, 而是编译器和处理器按照调用约定共同实现的机制。计算机的工作可以沿着指令、寄存器、内存和栈的状态变化逐步追踪, 高级语言的抽象最终都会落实为处理器执行的一条条指令。